

Nüfus ve Sosyal Araştırmalar Enstitüsü - Yapay Zeka Modülü : Makine Öğrenmesi - 6

İlgili notlar hazırlanırken **Sayın Ercan Atam** hocamızın notlarından faydalanılmıştır. Verdiği eğitim ve güzel anlatımı için teşekkürlerimi sunarım.

1. Boyut İndirgeme ve Temel Bileşenler Analizi (PCA)

1.1 Boyut İndirgemeye Giriş ve Motivasyon

Makine öğrenmesi modelleri, eğitim verilerinden türetilen ve verilerin "özlü" temsilleri olan öznitelikleri (features) girdi olarak kullanır. Ancak öznitelik sayısının çok fazla olması algoritmik analiz süreçlerinde hesaplama maliyetlerini artırır. Boyut indirgeme (dimensionality reduction), yüksek boyutlu bir veri kümesini, temel yapısını ve ayırt edici bilgisini koruyacak şekilde daha düşük boyutlu bir özellik uzayında temsil etme sürecidir.

İstatistiksel modellemelerde ve veri madenciliğinde boyut indirgemeye şu sebeplerle ihtiyaç duyarız:

- Zaman karmaşıklığını (time complexity) azaltır ve daha az parametre gerektirerek uzay karmaşıklığını (space complexity) düşürür.
- Özniteliğin gözlemlenmesi veya hesaplanması için katlanılan maliyetten tasarruf sağlar.
- Daha basit modeller, küçük boyutlu veri kümeleri üzerinde daha sağlam (robust) sonuçlar üretir.
- Veri, 2 veya 3 boyutta gösterildiğinde yapının, grupların ve aykırı değerlerin (outliers) görselleştirilmesine olanak tanır.

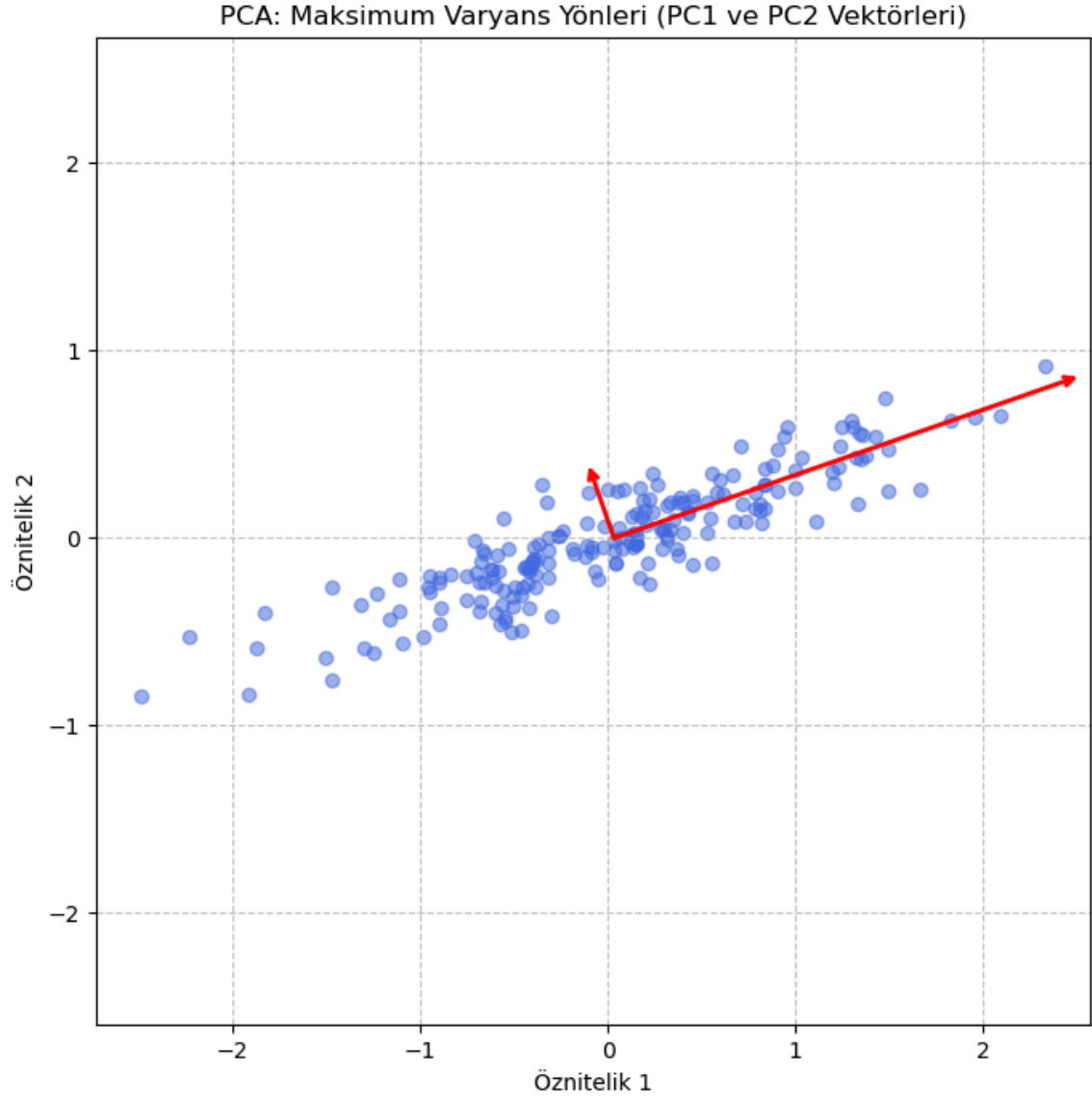
1.2 Temel Bileşen Analizi (PCA) Matematiksel Altyapısı

PCA, doğrusal boyut indirgemede $z = A * x$ şeklinde doğrusal bir dönüşüm kullanır. Yöntemin dayandığı ana ilke basittir: Daha fazla bilgi, daha büyük varyans anlamına gelir. PCA'nın amacı, bu maksimum varyansı sağlayacak birbirine dik (orthogonal) izdüşüm yönlerini ardışık olarak belirlemektir.

Matematiksel olarak x vektörünün w yönündeki izdüşümü $z = w^T * x$ ile ifade edilir. İzdüşüm varyansı ise, veri kovaryans matrisi S kullanılarak $\sigma^2 = w^T * S * w$ şeklinde hesaplanır. Birinci temel bileşen, $w_1^T * w_1 = 1$ kısıtı altında bu varyansı maksimize eden w_1 vektörüdür. Bu problem bir Lagrange optimizasyon problemi olarak çözüldüğünde elde edilen $S * w_1 = \alpha * w_1$ eşitliği, birinci temel bileşenin S matrisinin en büyük özdeğerine karşılık gelen özvektörü olduğunu kesinleştirir. Benzer bir türev işlemiyle, ikinci temel

bileşenin de kovaryans matrisi S 'in en büyük ikinci özdeğerine karşılık gelen özvektörü olduğu kanıtlanabilir.

Kritik Uygulama Notu: Derste kovaryans matrisi üzerinden özdeğer ayrışımı anlatılsa da, pratik veri analizi süreçlerinde (örneğin Python'da scikit-learn ile model kurarken) S matrisi doğrudan hesaplanmaz. Yazılım arka planda veri matrisi üzerinde doğrudan Tekil Değer Ayrışımı (Singular Value Decomposition - SVD) algoritmasını uygular. SVD, kovaryans matrisini açıkça hesaplamadan temel bileşenleri bularak bilgisayardaki kayan nokta (floating-point) yuvarlama hatalarını minimize eder ve işlemleri sayısal olarak daha kararlı (numerically stable) hale getirir.



1.3 Pratik PCA Prosedürü ve Model Seçimi

Teorik altyapının algoritmik prosedüre dökülmüş hali şu 4 adımı izler:

1. Eğitim verilerinden veri kovaryans matrisi S hesaplanır.
2. S matrisinin en büyük k özdeğerine karşılık gelen k tane özvektörü bulunur.

3. İzdüşüm matrisi oluşturulur: $W = [w_1 \ w_2 \ \dots \ w_k]$.
4. Herhangi bir x vektörü, $z = W^T * (x - \bar{x})$ ile (izdüşümden önce merkezleştirilerek) eşlenir.

İndirgenecek boyut olan k sayısının optimum seçimi için "Açıklanan Varyans Oranı" (Proportion of Variance - PoV) hesaplanır: $PoV = (\lambda_1 + \lambda_2 + \dots + \lambda_k) / (\lambda_1 + \lambda_2 + \dots + \lambda_d)$

Pratikte analiz yapılırken genellikle $PoV > 0.9$ (varyansın %90'ının açıklanması) olduğunda bileşen eklemeye son verilir. Ayrıca Scree grafiği üzerinde k değerine karşı özdeğerlerin (veya PoV 'nin) çizimi yapıp, grafikteki "dirsek" noktasında kesim yapılması da önerilen bir yöntemdir.

Önemli bir istatistiksel detay olarak; eğer bir öz niteliğin varyansı diğerlerine göre belirgin biçimde daha büyükse, bu durum varyansı büyük olan öz niteliğin birinci temel bileşene baskın şekilde yön vermesine sebep olur. Bu nedenle, PCA modellemesine geçilmeden önce veriyi standartlaştırarak (preprocessing adımıyla) her boyutun ortalamasını 0 ve varyansını 1 yapmak zorunlu bir uygulamadır. Ayrıca PCA varyansı maksimize etmeye odaklandığı için aykırı değerlere (outliers) son derece duyarlıdır; veri setindeki merkezden uzak birkaç nokta, özvektörler üzerinde ciddi sapmalara neden olabilir.

JupyterLab üzerinde konuyu pratik etmek için örnek bir PCA bloğu

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_breast_cancer
```

Veriyi içeri aktarma

```
veri = load_breast_cancer()
X = veri.data
```

Teoride belirtilen kritik ön işleme (Ortalama=0, Varyans=1)

```
olceklendirici = StandardScaler()
X_olcekli = olceklendirici.fit_transform(X)
```

Varyansın %90'ını (PoV > 0.9) açıklayacak kadar bileşen seçimi

```
pca_modeli = PCA(n_components=0.90)
X_indirgenmis = pca_modeli.fit_transform(X_olcekli)
```

```
print(f"Original Boyut (d): {X.shape[1]}")
print(f"İndirgenmiş Boyut (k): {X_indirgenmis.shape[1]}")
print(f"Açıklanan Kümülatif Varyans (PoV): {np.sum(pca_modeli.explained_variance_ratio_):.2f}")
```

Scree Grafiği ile Dirsek Noktasının Gözlemlenmesi

```
pca_tum = PCA().fit(X_olcekli)
plt.plot(np.cumsum(pca_tum.explained_variance_ratio_), marker='o',
linestyle='--')
```

```
plt.xlabel('Seçilen Temel Bileşen Sayısı (k)')
plt.ylabel('Kümülatif Açıklanan Varyans (PoV)')
plt.title('PCA Scree Grafiği')
plt.grid(True)
plt.show()
```

2. Doğrusal Ayırt Edici Analiz (Linear Discriminant Analysis - LDA)

2.1 LDA'ya Giriş ve Temel Amacı

Temel Bileşen Analizi (PCA) verideki varyansı maksimize etmeye odaklanan denetimsiz (unsupervised) bir yöntemken; Doğrusal Ayırt Edici Analiz (LDA), sınıflandırma problemlerinde boyut indirgeme için kullanılan **denetimli (supervised)** bir yöntemdir.

LDA'nın temel amacı; veri setini daha düşük boyutlu bir uzaya yansıtırken (projection), sınıflar (kategoriler) arasındaki ayrımı (ayrılabilirliği) maksimize edecek doğrusal bir eşleme bulmaktır. Bu işlemi yaparken sınıf etiketleri (labels) bilinir ve izdüşüm mekanizması tamamen bu bilgi üzerine kurulur.

2.2 Fisher'ın Doğrusal Ayırt Edicisi ve Matematiksel Altyapı

LDA, sınıfların iyi ayrılabilmesi için iki temel istatistiksel koşulu aynı anda sağlamaya çalışır:

- Sınıflar Arası Saçılımı (Between-class scatter) Maksimize Etmek:** Farklı sınıfların ortalamalarının (merkezlerinin) birbirinden mümkün olduğunca uzak olması.
- Sınıf İçi Saçılımı (Within-class scatter) Minimize Etmek:** Aynı sınıfa ait örneklerin izdüşümden sonra kendi ortalamaları etrafında mümkün olduğunca dar bir bölgede, sıkıca toplanması.

İki sınıflı ($K = 2$) bir problemde, x vektörünün w üzerine izdüşümü $z = w^T x$ ile d boyutundan 1 boyuta indirgenir. Optimizasyon problemi, Fisher'ın doğrusal ayırt edicisi olarak bilinen şu amaç fonksiyonunu ($J(w)$) maksimize etmektir:

$$J(w) = \frac{(m_1 - m_2)^2}{s_1^2 + s_2^2} = \frac{w^T S_B w}{w^T S_W w}$$

Burada:

- S_B : Sınıflar arası saçılım matrisi (Between-class scatter matrix)
- S_W : Sınıf içi saçılım matrisi (Within-class scatter matrix)

Bu fonksiyonun w 'ye göre türevi alınıp sıfıra eşitlendiğinde, boyut indirgeme için kullanılacak optimal izdüşüm vektörü şu şekilde bulunur:

$$w = S_W^{-1}(m_1 - m_2)$$

2.3 PCA ve LDA Karşılaştırması

Sınıflandırma problemlerinde izdüşümün asıl amacı sınıflar arası ayrımı maksimize etmek olduğunda PCA başarısız olabilir. Çünkü PCA, sınıf etiketlerini görmezden gelerek sadece verinin en çok dağıldığı yöne (maksimum varyans) odaklanır. Veri setindeki genel varyansın en yüksek olduğu yön, sınıfları birbirinden ayıran en iyi yön olmayabilir. LDA ise sınıf etiketlerini denkleme katarak, doğrudan sınıfları birbirinden en iyi ayıran eksenini bulur.

Ek Bilgi: İstatistiksel Analizlerde LDA ve Varyans Analizi (ANOVA) İlişkisi Pratik istatistiksel analizlerde, varyans analizi (ANOVA) veya onun çok değişkenli versiyonu olan MANOVA uygulanırken arka planda yatan matematiksel mantık ile LDA'nın altyapısı neredeyse aynıdır. İstatistiksel paket programlarında (örneğin SPSS) grup ortalamaları arasındaki farkların anlamlılığını test ederken hesaplanan "Gruplar Arası (Between)" ve "Grup İçi (Within)" kareler toplamı matrisleri, tam olarak LDA'daki S_B ve S_W matrisleridir. Bu nedenle LDA, sadece bir makine öğrenmesi boyut indirgeme aracı değil, aynı zamanda bağımlı kategorik değişkenlerin bağımsız sürekli değişkenler tarafından nasıl açıklandığını gösteren köklü bir çok değişkenli istatistiksel testtir.

Pratik LDA Uygulaması (Scikit-Learn)

Aşağıdaki örnekte, PCA ve LDA'nın aynı veri seti üzerindeki davranış farkını ve LDA'nın sınıflandırma performansını artıracak öznitelikleri nasıl çıkardığını inceleyebiliriz.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# 1. Veri Yükleme (Örnek Şarap Veri Seti - 3 Sınıflı)
veri = load_wine()
X = veri.data
y = veri.target # LDA için sınıf etiketleri (labels) şarttır!

# 2. Ön İşleme
olceklendirici = StandardScaler()
X_olcekli = olceklendirici.fit_transform(X)

# 3. Modellerin Kurulması
# İki bileşene indirelim ki 2 boyutlu grafikte görselleştirebileyim
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_olcekli)

lda = LinearDiscriminantAnalysis(n_components=2)
X_lda = lda.fit_transform(X_olcekli, y) # Dikkat: LDA 'y' (etiket) bekler!

# 4. Çıktıların Karşılaştırmalı Görselleştirilmesi
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
```

```

renkler = ['navy', 'turquoise', 'darkorange']
for renk, i, hedef_adi in zip(renkler, [0, 1, 2], veri.target_names):
    ax1.scatter(X_pca[y == i, 0], X_pca[y == i, 1], color=renk, alpha=.8,
lw=2, label=hedef_adi)
    ax2.scatter(X_lda[y == i, 0], X_lda[y == i, 1], color=renk, alpha=.8,
lw=2, label=hedef_adi)

ax1.set_title('PCA İzdüşümü (Denetimsiz)')
ax2.set_title('LDA İzdüşümü (Denetimli)')
ax1.legend(loc='best', shadow=False, scatterpoints=1)
ax2.legend(loc='best', shadow=False, scatterpoints=1)
plt.show()

```

3. Destek Vektör Makineleri (Support Vector Machines - SVM)

3.1 Temel Kavramlar: Ayırıcı Hiper Düzlem ve Marj (Margin)

Destek Vektör Makineleri (SVM), temel olarak sınıfları birbirinden ayıran "en iyi" doğruyu (veya çok boyutlu uzayda hiper düzlemi) bulmaya çalışan denetimli bir makine öğrenmesi algoritmasıdır.

Sınıflandırma problemlerinde veriyi ayırabilecek sonsuz sayıda doğru çizilebilir. SVM'in felsefesi şudur: Sadece veriyi ayırmakla kalmayıp, aynı zamanda sınırdaki bulunan veri noktalarına (her iki sınıf için de) en uzak olanı, yani "**en kalın tampon bölgeye (margin)**" sahip olanı seç.

- Ayırıcı hiper düzlem denklemi: $w^T \cdot x + b = 0$
- Bir hiper düzlemin kalınlığı (Marj): $1 / ||w||$

Daha kalın bir tampon bölge, modelin gürültülü (noisy) verilere ve ölçüm hatalarına karşı çok daha dayanıklı (robust) olmasını sağlar.

3.2 Sert Marjlı (Hard-Margin) SVM ve Destek Vektörleri

Sert-marjlı SVM, verilerin doğrusal olarak "tamamen ve kusursuzca" ayrılabilirliğini varsayar. Hiçbir sınıflandırma hatasına izin vermeden maksimum marjlı ayırıcı hiper düzlemi bulmaya çalışır.

Optimizasyon problemi şu şekilde ifade edilir: Amaç: $\min (1/2) \cdot w^T \cdot w$ Kısıt (Subject to): $y^n \cdot (w^T \cdot x^n + b) \geq 1$

Bu denklemdeki ≥ 1 kısıtı, tüm veri noktalarının marjın dışında veya tam sınırında olması gerektiğini belirtir. **Destek Vektörleri (Support Vectors):** Tam olarak marjın sınır çizgisi üzerinde yer alan veri noktalarıdır ($y^n \cdot (w^T \cdot x^n + b) = 1$). Bu noktalar, adeta hiper

düzlemi "destekler" ve tampon bölgenin daha fazla genişlemesini engellerler. İlginç bir istatistiksel özellik olarak; destek vektörü olmayan diğer tüm veri noktalarını veri setinden çıkarsanız bile, optimum ayırıcı hiper düzlem ve marj çizgisi kesinlikle değişmez. Model sadece bu sınır noktalarına (destek vektörlerine) göre inşa edilir.

3.3 Yumuşak Marjlı (Soft-Margin) SVM ve Hata Toleransı

Gerçek dünya verileri nadiren kusursuzca ayrılabilir formdadır. Verilerde iç içe geçmeler veya aykırı değerler olduğunda Hard-Margin SVM çözüm bulamaz. Bu sorunu aşmak için "Yumuşak Marj" kavramı geliştirilmiştir.

Yumuşak marj, modelin bazı veri noktalarında sınıflandırma hatası yapmasına veya noktaların marjın içine girmesine esneklik tanır. Bu durum optimizasyon denkleminde bir "yumuşak hata (slack variable)" parametresi (ξ) eklenerek yapılır: Amaç: $\min (1/2) * w^T * w + C * \sum \xi_i$

Ekstra Akademik Bilgi (Varyans-Yanlılık Dengesi): Denklemdaki **C (Regülerizasyon Parametresi)** makine öğrenmesindeki en kritik hiperparametrelerden biridir.

- C değeri çok büyük seçilirse: Algoritma hatalara karşı çok tahammülsüz olur (Hard-margin'e yaklaşır). Marj daralır, model eğitim verisini ezberleyebilir (Overfitting / Yüksek Varyans).
- C değeri çok küçük seçilirse: Algoritma hatalara çok toleranslı hale gelir. Marj genişler ancak sınıflandırma başarısı düşebilir (Underfitting / Yüksek Yanlılık).

3.4 Çekirdek Numarası (Kernel Trick) ve Doğrusal Olmayan Dönüşümler

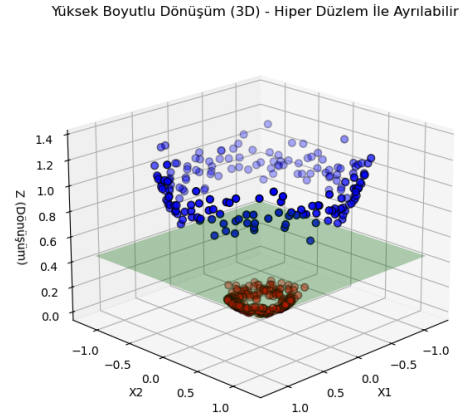
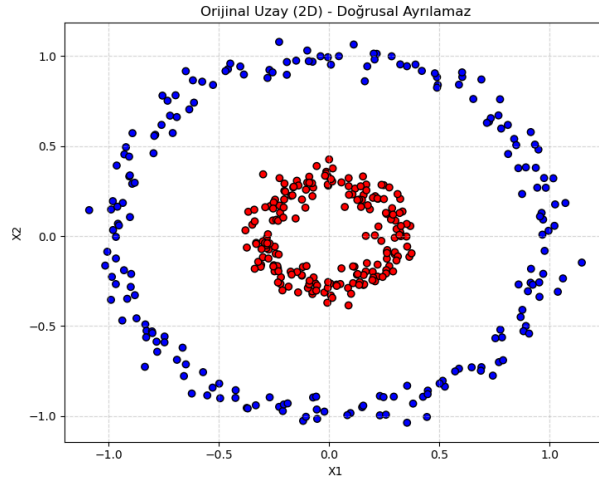
Eğer veri orijinal uzayında hiçbir şekilde doğrusal bir doğru/düzlem ile ayrılıyorsa ne yapılır? Çözüm, veriyi doğrusal olarak ayrılabilir hale gelebileceği çok daha yüksek (belki de sonsuz) boyutlu bir uzaya dönüştürmektir (Transformasyon: $z^n = \Phi(x^n)$).

Ancak, her bir veri noktasını matematiksel olarak yüksek boyutlara taşımak devasa bir işlemci gücü gerektirir. İşte bu noktada "**Kernel Trick (Çekirdek Numarası)**" devreye girer. Çekirdek fonksiyonları, veriyi gerçekten yüksek boyutlu uzaya dönüştürmeden, o uzaydaki "iç çarpım (inner product)" sonucunu orijinal uzayda hesaplayabilen mucizevi matematiksel kestirmelerdir. Formülü: $K(x, x') = \Phi(x)^T * \Phi(x')$

Sık kullanılan çekirdekler:

1. **Polinom Çekirdeği (Polynomial Kernel):** Eğrisel sınırları yakalamak için kullanılır.
2. **Gaussian-RBF Çekirdeği:** En popüler olanıdır. Veriyi sonsuz boyutlu bir uzaya taşıyormuş gibi işlem yaparak çok karmaşık, dairesel ve iç içe geçmiş veri kümelerini

kusursuzca ayırabilir.



Pratik SVM Uygulaması (Scikit-Learn)

Aşağıdaki kod bloğu, Doğrusal (Linear) ve Doğrusal Olmayan (RBF) Kernel kullanılarak SVM'in karar sınırlarının nasıl değiştiğini analiz etmen için hazırlanmıştır.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.datasets import make_circles
from sklearn.preprocessing import StandardScaler

# 1. Doğrusal olarak ayırlamayan (iç içe geçmiş halkalar) sentetik veri üretimi
X, y = make_circles(n_samples=300, factor=0.3, noise=0.1, random_state=42)

# SVM için ölçeklendirme hayati önem taşır (uzaklık temelli bir algoritma olduğu için)
scaler = StandardScaler()
X_olcekli = scaler.fit_transform(X)

# 2. İki farklı SVM Modeli Kurulumu
# Model 1: Doğrusal (Linear) Çekirdek - C parametresi 1.0 (Standart)
svm_dogrusal = SVC(kernel='linear', C=1.0)
svm_dogrusal.fit(X_olcekli, y)

# Model 2: Gaussian RBF Çekirdeği (Karmaşık sınırlar için)
svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale')
svm_rbf.fit(X_olcekli, y)

# 3. Sonuçların Görselleştirilmesi (Karar Sınırlarının Çizimi)
def karar_siniri_ciz(model, ax, baslik):
    xx, yy = np.meshgrid(np.linspace(-2.5, 2.5, 100), np.linspace(-2.5, 2.5, 100))
    Z = model.decision_function(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
```



```

ax.scatter(X_olcekli[:, 0], X_olcekli[:, 1], c=y, cmap='bwr',
edgecolors='k', s=40)
# Karar sınırını ve marjları (tampon bölgeyi) çiz
ax.contour(xx, yy, Z, colors='k', levels=[-1, 0, 1], alpha=0.5,
linestyles=['--', '-', '--'])
ax.set_title(baslik)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Doğrusal kernel bu veriyi ayırmakta başarısız olacaktır (Çünkü veri
daireesel)
karar_siniri_ciz(svm_dogrusal, ax1, 'Doğrusal (Linear) SVM')

# RBF kernel Kernel Trick sayesinde veriyi kusursuzca ayıracaktır
karar_siniri_ciz(svm_rbf, ax2, 'Gaussian RBF SVM (Kernel Trick)')

plt.tight_layout()
plt.show()

```

4. Uçtan Uca Örnek Çözüm: PCA ve SVM Entegrasyonu

4.1 İleri Seviye Makine Öğrenmesi Boru Hattı (Pipeline) Mantiği

Gerçek dünya projelerinde algoritmalar genellikle izole edilerek değil, birbirini tamamlayacak şekilde bir akış (pipeline) içerisinde kullanılır. Yüksek boyutlu bir veri setinde doğrudan Destek Vektör Makineleri (SVM) kullanmak, "boyutun laneti" (curse of dimensionality) nedeniyle modelin çok fazla gürültüyü ezberlemesine (overfitting) ve hesaplama maliyetinin üstel olarak artmasına yol açabilir.

Bu örnek çözümde, 30 farklı medikal özneliğe sahip göğüs kanseri veri setini ele alacağız. Veriyi önce standartlaştıracak, ardından Temel Bileşenler Analizi (PCA) ile varyansın büyük kısmını koruyarak boyutunu küçültecek ve elde edilen bu gürültüden arındırılmış (özet) veri üzerinde Gaussian RBF çekirdekli bir SVM sınıflandırıcısı eğiteceğiz.

Ek Bilgi (Veri Sızıntısı - Data Leakage Koruması): Makine öğrenmesi uygulamalarında yapılan en kritik istatistiksel hatalardan biri, ölçeklendirme (StandardScaler) ve boyut indirgeme (PCA) işlemlerini tüm veri setine tek seferde uygulamaktır. Bu yaklaşım, test setindeki bilgilerin eğitim aşamasına "sızmasına" neden olur. Bu nedenle pratik örnekte görüleceği üzere; `fit` (öğrenme) işlemi **sadece eğitim setine** uygulanmalı, test seti ise eğitim setinden öğrenilen parametrelerle sadece `transform` (dönüştürme) işlemine tabi tutulmalıdır.

4.2 Örnek Çözüm Kod Blokları ve Değerlendirme

Aşağıdaki kodu JupyterLab üzerinde yeni bir hücrede çalıştırarak, tüm bu teorik konseptlerin bir arada nasıl harika bir performans sergilediğini görebilirsin.

```
In [1]: # 0. Kütüphanelerin yüklenmesi
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix

# 1. Veri Yükleme
veri = load_breast_cancer()
X = veri.data
y = veri.target

# 2. Eğitim ve Test Setlerine Ayırma (%80 Eğitim, %20 Test)
X_egitim, X_test, y_egitim, y_test = train_test_split(X, y, test_size=0.2, random_s

# 3. Ön İşleme (Standardizasyon)
# DİKKAT: Veri sızıntısını önlemek için 'fit' sadece eğitim verisine yapılır!
olceklendirici = StandardScaler()
X_egitim_olcekli = olceklendirici.fit_transform(X_egitim)
X_test_olcekli = olceklendirici.transform(X_test)

# 4. Boyut İndirgeme (PCA)
# Orijinal 30 öz niteliği varyansın %95'ini açıklayacak şekilde indirgiyoruz
pca = PCA(n_components=0.95)
X_egitim_pca = pca.fit_transform(X_egitim_olcekli)
X_test_pca = pca.transform(X_test_olcekli)

print(f"Orijinal Öz nitelik Sayısı: {X_egitim.shape[1]}")
print(f"PCA Sonrası Öz nitelik Sayısı: {X_egitim_pca.shape[1]} (Varyansın %95'i koru

# 5. Destek Vektör Makinesi (SVM) Modeli Eğitimi
# RBF çekirdekli, C=1.0 (Yumuşak Marj) parametresi ile model kurulumu
svm_modeli = SVC(kernel='rbf', C=1.0, gamma='scale')
svm_modeli.fit(X_egitim_pca, y_egitim)

# 6. Tahmin ve Değerlendirme
y_tahmin = svm_modeli.predict(X_test_pca)

# Sonuçların Raporlanması
print("--- SVM Sınıflandırma Raporu ---")
print(classification_report(y_test, y_tahmin, target_names=veri.target_names))

# Karmaşıklık Matrisi (Confusion Matrix) Görselleştirmesi
plt.figure(figsize=(6, 4))
sns.heatmap(confusion_matrix(y_test, y_tahmin), annot=True, fmt='d', cmap='Blues',
            xticklabels=veri.target_names, yticklabels=veri.target_names)
plt.title('SVM Sınıflandırma Sonuçları (PCA Entegrasyonlu)')
plt.ylabel('Gerçek Sınıflar')
```

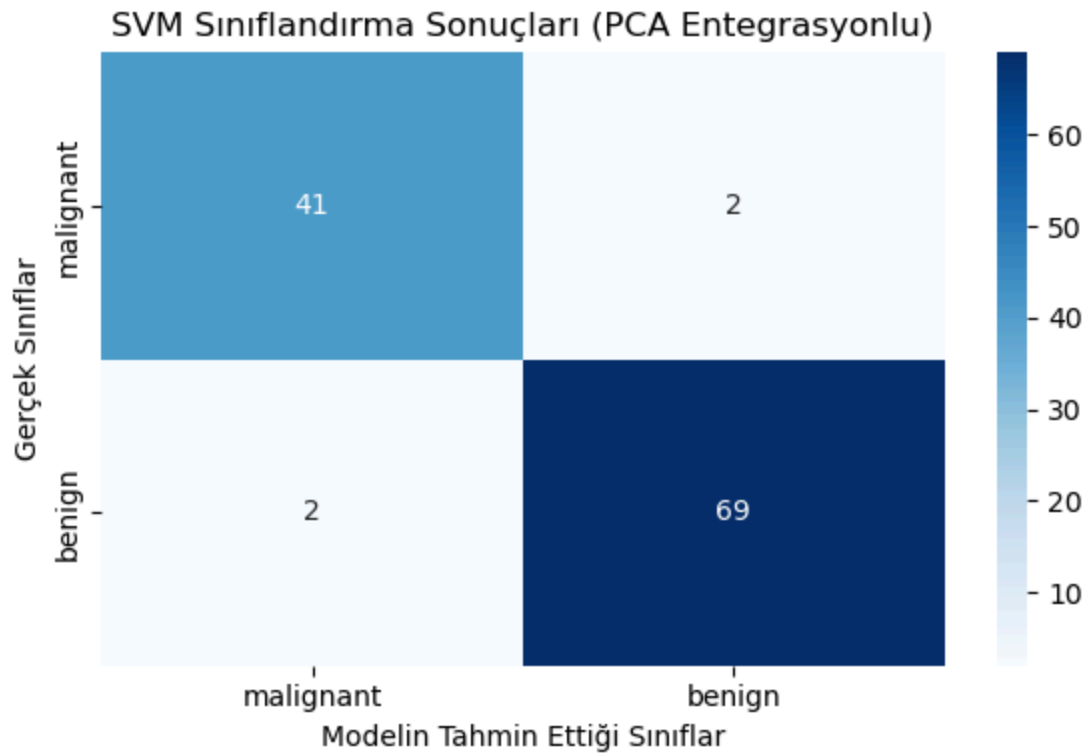
```
plt.xlabel('Modelin Tahmin Ettiği Sınıflar')
plt.tight_layout()
plt.show()
```

Original Öznitelik Sayısı: 30

PCA Sonrası Öznitelik Sayısı: 10 (Varyansın %95'i korundu)

--- SVM Sınıflandırma Raporu ---

	precision	recall	f1-score	support
malignant	0.95	0.95	0.95	43
benign	0.97	0.97	0.97	71
accuracy			0.96	114
macro avg	0.96	0.96	0.96	114
weighted avg	0.96	0.96	0.96	114



5. İleri Seviye Ek Not: Model Optimizasyonu ve Modern Yapay Zeka Bağlantısı (Ders Dışı Ekstra)

5.1 Hiperparametre Optimizasyonu (GridSearchCV)

Önceki SVM örneklerimizde regülerizasyon parametresi olan C ve çekirdek katsayısı olan γ değerlerini (örneğin $C = 1.0$, `gamma='scale'`) sabit olarak belirledik. Ancak profesyonel ve akademik veri analizi projelerinde bu hiperparametreleri deneme yanılma ile veya varsayılan (default) değerlerle bırakmak, modelin varyans-yanlılık (bias-variance) dengesini bozarak "Aşırı Öğrenme" (Overfitting) veya "Eksik Öğrenme" (Underfitting) sorunlarına yol açar.

Bunun istatistiksel ve algoritmik çözümü **Çapraz Doğrulama (Cross-Validation)** destekli **Izgara Aramasıdır (Grid Search)**. Bu yöntem, belirlediğiniz parametre uzayındaki tüm olası kombinasyonları sistematik olarak dener, veriyi farklı parçalara bölerek test eder ve en yüksek performansı veren modeli otomatik olarak seçer.

```
In [2]: # Model Optimizasyonu (GridSearchCV) İçin Örnek Python Kodu
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC

# 1. Denenmesini istediğimiz parametre uzayının (Grid) tanımlanması
parametre_uzayi = {
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001],
    'kernel': ['rbf', 'linear'] # Hangi çekirdeğin daha iyi olduğunu da algoritmaya
}

# 2. Çapraz doğrulama (cv=5) ile optimizasyon modelinin kurulması
# verbose=1 ile eğitim sırasındaki ilerlemeyi konsolda görebiliriz
grid_aramasi = GridSearchCV(SVC(), parametre_uzayi, refit=True, verbose=1, cv=5)

# 3. Modelin eğitilmesi (X_egitim_pca ve y_egitim verilerini Bölüm 4'ten varsayıyor)
grid_aramasi.fit(X_egitim_pca, y_egitim)

# 4. En iyi parametrelerin ve skorun raporlanması
print("İstatistiksel Olarak En İyi Parametreler:", grid_aramasi.best_params_)
print("Modelin En İyi Çapraz Doğrulama Skoru:", grid_aramasi.best_score_)

# Tahmin yaparken artık doğrudan grid_aramasi nesnesini kullanabiliriz (En iyi mode
y_tahmin_optimum = grid_aramasi.predict(X_test_pca)
```

Fitting 5 folds for each of 32 candidates, totalling 160 fits

İstatistiksel Olarak En İyi Parametreler: {'C': 0.1, 'gamma': 1, 'kernel': 'linear'}

Modelin En İyi Çapraz Doğrulama Skoru: 0.9758241758241757

5.2 Geleceğe Bakış: PCA ve Otokodlayıcılar (Autoencoders) Arasındaki Matematiksel Bağlantı

Makine öğrenmesi derslerinde doğrusal boyut indirgeme için kullanılan **PCA**, günümüzdeki devasa Derin Öğrenme (Deep Learning) mimarilerinin kavramsal atalarından biridir.

Günümüzde yapay zeka araştırmacıları, karmaşık boyut indirgeme ve öznetelik çıkarımı işlemleri için **Otokodlayıcı (Autoencoder)** adı verilen yapay sinir ağlarını kullanmaktadır. Matematiksel olarak kanıtlanmış şaşırtıcı bir gerçek şudur: Eğer tek gizli katmanlı (single hidden layer) bir Otokodlayıcı ağı kurar ve aktivasyon fonksiyonu olarak doğrusal (linear) bir fonksiyon seçerseniz, bu yapay sinir ağının geri yayılım (backpropagation) ile öğreneceği "darboğaz (bottleneck)" ağırlıkları, PCA'nin hesapladığı Temel Bileşenler ile tamamen aynı özellik uzayını (feature subspace) kapsayacaktır.

Bu durum, klasik doğrusal cebir tabanlı istatistik ile modern yapay sinir ağlarının birbirlerinden kopuk olmadığını; aksine mükemmel bir evrimsel bütünlük içinde olduklarını kanıtlar.

-Memreu